دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه هجدهم: درخت‌ها (تعاریف اولیه)

دکتر شهرام گلزاری

پاییز ۱۴۰۵

اهداف جلسه

در پایان این جلسه انتظار داریم که:

- درخت آزاد را تعریف کنیم و نشان دهیم در درخت با n گره دقیقاً $n-1$ یال وجود دارد.
- درخت ریشه‌دار و تعریف بازگشتی آن را بشناسیم و اصطلاحات ریشه، برگ، برادر/خواهر، گره داخلی، ارتفاع، سطح و درخت kتایی را به کار ببریم.
- پیمایش‌های پیش‌ترتیب، میان‌ترتیب و پس‌ترتیب را روی یک درخت اجرا کنیم.
- درخت دودویی معادل یک درخت ریشه‌دار را بسازیم و رابطه پیمایش‌های آن دو را بیان کنیم.
- اعمال اصلی روی درخت و شبه‌کد آن‌ها را بنویسیم.

درخت آزاد

چرا درخت؟

«درخت یک مدل مناسب برای نمایش بسیاری از مفاهیم، پدیده‌ها و رابطه‌ی بین آن‌هاست. مثلاً، درخت فامیلی، سلسله‌مراتب اداری، عبارت‌های ریاضی و مراحل مختلف یک بازی به‌خوبی با انواع درخت‌ها نشان داده می‌شوند. به‌صورت انتزاعی درخت‌ها گراف‌های خاصی هستند که در مورد ویژگی‌های آن‌ها نتایج نظری زیادی موجود است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

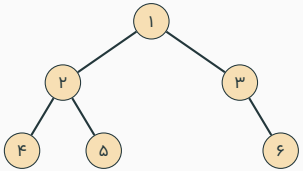
تعریف درخت آزاد

تعریف ۱. درخت آزاد

«درخت یک گراف هم‌بند و بدون دور است که به آن «درخت آزاد» هم می‌گوییم. رأس‌های این گراف گره نامیده می‌شوند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

- هم‌بند: بین هر دو گره مسیر وجود دارد.
- بدون دور: هیچ مسیر بسته‌ای ندارد.
- بین هر دو گره تنها یک مسیر ساده وجود دارد.



درخت آزاد با ۶ گره و ۵ یال

خواص درخت آزاد

نکته ۴-۱

«یک گراف درخت است اگر بین هر دو رأس آن تنها یک مسیر وجود داشته باشد. چنین درختی با n گره دارای $n - ۱$ یال است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

لم ۴-۱

«اگر E تعداد یال‌های یک درخت آزاد و V تعداد گره‌های آن باشد، آنگاه

$$E = V - ۱.$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

اثبات با استقرا.

- برای $V = ۱$ بدیهی است که $E = ۰$.
- برای $V > ۱$ حتماً گره‌ای با درجهٔ ۱ وجود دارد؛ وگرنه دور داریم.
- این گره و یال متصل به آن را حذف می‌کنیم. درختی با $V - ۱$ گره و $E - ۱$ یال به دست می‌آید.
- طبق فرض استقرا $E - ۱ = (V - ۱) - ۱$، پس $E = V - ۱$.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

درخت ریشه‌دار

تعریف ۲. درخت جهت‌دار

«یک «درخت جهت‌دار» مجموعه‌ای از عناصر است که رابطه‌ی «پدر-فرزندی» بین آن‌ها برقرار است به‌طوری که هر عنصر به‌جز «ریشه» دقیقاً یک پدر داشته باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

«به‌عبارت دیگر، اگر بر روی یال‌های یک درخت آزاد جهت قرار دهیم به‌طوری که درجه‌ی ورودی هر گره به‌جز گره ریشه ۱ باشد، درخت جهت‌دار ایجاد شده است. جهت‌ها نشان‌دهنده‌ی رابطه‌ی «پدر-فرزندی» است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰

تعریف بازگشتی درخت ریشه‌دار

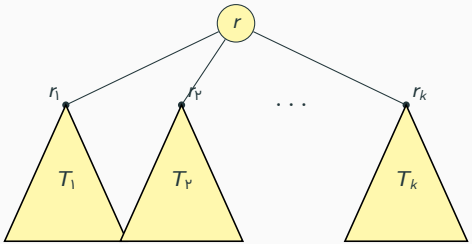
تعریف ۳

«درخت ریشه‌دار» مطابق زیر تعریف می‌شود:

۱. یک گره به‌تنهایی یک درخت ریشه‌دار است.

۲. اگر k درخت ریشه‌دار مستقل T_1 تا T_k به‌ترتیب با ریشه‌های r_1 تا r_k داشته باشیم، می‌توانیم یک درخت ریشه‌دار بزرگ‌تر به‌نام T با ریشه‌ی r بسازیم به‌طوری که r پدر r_1 تا r_k باشد. در این‌صورت T_1 تا T_k را «زیردرخت»های T می‌گوییم.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۰



تعاریف اولیه ۱

- **ریشه** (root): گره‌ای که دارای پدر نیست. در هر درخت جهت‌دار یکی است.
- **برگ** (leaf): گره بدون فرزند.
- **برادر** (sibling): گره‌هایی که یک پدر دارند.
- **گره داخلی** (interior node): گره غیر برگ.
- **ارتفاع گره** v: طول بزرگ‌ترین مسیر از v به یک برگ در زیردرختش.
- **ارتفاع درخت:** ارتفاع ریشه.
- **سطح یا عمق گره:** طول مسیر از ریشهٔ درخت به آن گره.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۱

تعاریف اولیه ۲

- **درخت متوازن** (balanced tree)**:** درختی که سطح برگ‌های آن حداکثر یک واحد با هم اختلاف داشته باشد.
- **درخت کاملاً متوازن:** درختی که سطح برگ‌های آن یکی است.
- **درخت kتایی:** درختی که بیشینه‌ی تعداد فرزندان یک گره k باشد.
- **درخت kتایی کامل:** درخت متوازنی که در آن تعداد فرزندان هر گره برابر k یا صفر (فقط برای برگ‌ها) باشد.
- **درخت مرتب** (ordered tree)**:** درختی که ترتیب فرزندان هر گره مشخص است.
- **درخت برچسب‌دار** (labeled tree)**:** درختی که هر گره آن یک برچسب دارد.
- **درخت دودویی** (binary tree)**:** درخت مرتبی که هر عنصر آن حداکثر دو فرزند به نام‌های فرزند چپ و راست دارد.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۱

تعاریف اولیه ۳

- **اولاد یک گره** v**:** گره‌های موجود در زیردرختی به ریشه‌ی v. هر گره اولاد خودش هم است.
- **اجداد یک گره** v**:** گره‌های موجود در مسیر از ریشه به v. هر گره جزو اجداد خودش است.
- **اولاد واقعی:** اولاد v به‌جز v.
- **اجداد واقعی:** اجداد v به‌جز v.
- **درخت پُر** (full tree)**:** درخت کامل و کاملاً متوازن.
- **جنگل** (forest)**:** تعدادی درخت.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۲

پیمایش درخت‌ها

سه پیمایش اصلی

فرض کنید درخت ریشه‌دار و مرتب T با ریشه‌ی r و k زیردرخت T_1 تا T_k داریم. می‌خواهیم با حرکت روی یال‌ها همهٔ گره‌ها را هرکدام یک‌بار ملاقات کنیم. به این کار پیمایش می‌گوییم.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۳

- **پیش‌ترتیب:**

$$\text{Preorder}(T) = r, \text{Preorder}(T_1), \ldots, \text{Preorder}(T_k).$$

- **میان‌ترتیب:**

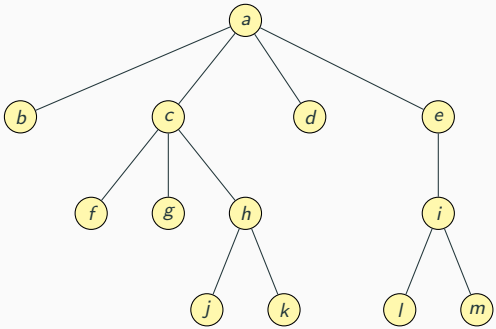
$$\text{Inorder}(T) = \text{Inorder}(T_1), r, \text{Inorder}(T_2), \ldots, \text{Inorder}(T_k).$$

- **پس‌ترتیب:**

$$\text{Postorder}(T) = \text{Postorder}(T_1), \ldots, \text{Postorder}(T_k), r.$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۳

مثال پیمایش



پیمایش	ترتیب
پیش‌ترتیب	*a*, *b*, *c*, *f*, *g*, *j*, *h*, *k*, *d*, *e*, *i*, *l*, *m*
میان‌ترتیب	*b*, *a*, *f*, *c*, *g*, *j*, *h*, *k*, *d*, *l*, *i*, *m*, *e*
پس‌ترتیب	*b*, *f*, *j*, *g*, *k*, *h*, *c*, *d*, *l*, *m*, *i*, *e*, *a*

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۱۸۳ و ۱۸۴

درخت دودویی معادل

ساخت درخت دودویی معادل

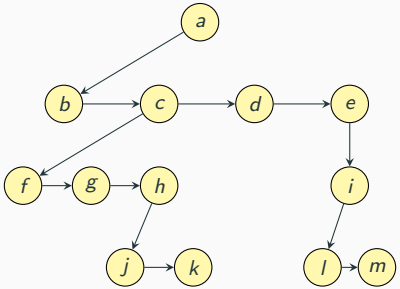
«هر درخت را می‌توان به‌صورت یک درخت دودویی درآورد که به آن درخت دودویی معادل هم با درخت اصلی می‌گوییم. درخت دودویی معادل در پیاده‌سازی درخت‌های کلی به‌کار می‌رود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۴

۱. اولین (سمت چپ‌ترین) فرزند هر گره در درخت اصلی، فرزند چپ آن گره در درخت دودویی معادل است.
۲. برادر سمت راست هر گره در درخت اصلی، فرزند راست آن در درخت معادل است.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۴

مثال درخت دودویی معادل



شکل ۴-۲۵ درخت دودویی معادل درخت شکل ۴-۲۴.

$$
\begin{aligned}
Preorder(T') &= a, b, c, f, g, h, i, j, d, e, k, l, m = Preorder(T) \\
Inorder(T') &= b, f, g, i, j, h, c, d, l, m, k, e, a = Postorder(T) \\
Postorder(T') &= j, i, h, g, f, m, l, k, e, d, c, b, a
\end{aligned}
$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۴

اعمال روی درخت

اعمال اصلی

«بر روی درخت T اعمال مختلفی را می‌توان انجام داد، ولی باید دست‌کم اعمال زیر در هر پیاده‌سازی فراهم شوند:»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۵

- $\textsc{Create}(T)$: درخت تهی T را ایجاد کن.
- $\textsc{Root}(T)$: ریشه‌ی درخت T را برگردان.
- $\textsc{Parent}(T, v)$: پدر گره v را برگردان.
- $\textsc{Left-Most-Child}(T, v)$: اولین فرزند گره v را برگردان.
- $\textsc{Right-Sibling}(T, v)$: برادر سمت راست v را برگردان.
- $\textsc{Size}(T)$: تعداد عناصر موجود در درخت را برگردان.
- $\textsc{IsEmpty}(T)$: مشخص کن که آیا درخت تهی است یا خیر.
- $\textsc{Element}(T, n)$: عنصر در گره n را برگردان.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۵ و ۱۸۶

پیمایش‌های پیاده‌سازی‌شده

```
POSTORDER(T, p)
1  if p = null
2     then return
3  n ← LEFT-MOST-CHILD(T, p)
4  while n ≠ null
5     do POSTORDER(T, n)
6        n ← RIGHT-SIBLING(T, n)
7  ELEMENT(T, p)
```

```
INORDER(T, p)
1  if p = null
2     then return
3  n ← LEFT-MOST-CHILD(T, p)
4  INORDER(T, n)
5  ELEMENT(T, p)
6  n ← RIGHT-SIBLING(T, n)
7  while n ≠ null
8     do INORDER(T, n)
9        n ← RIGHT-SIBLING(T, n)
```

```
PREORDER(T, p)
1  if p = null
2     then return
3  ELEMENT(T, p)
4  p ← LEFT-MOST-CHILD(T, p)
5  while p ≠ null
6     do PREORDER(T, p)
7        p ← RIGHT-SIBLING(T, p)
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۶

شمارش گره‌ها و ارتفاع

```
NodeHeight(T, p)
▷ ارتفاع گره p را در درخت T محاسبه می‌کند و آن‌را برمی‌گرداند
▷ از نحوه‌ی پیاده‌سازی درخت اطلاعاتی نداریم
1 if IsEmpty(T)
2    then return -1
3 height ← 0
4 p ← Left-Most-Child(T, p)
5 while p ≠ null
6    do height ← max{height, NodeHeight(T, p)}
7       p ← Right-Sibling(T, p)
8 return height + 1
```

```
CountNodes(T, p)
▷ تعداد گره‌های موجود در درخت T با ریشه‌ی p را می‌شمارد
1 if T = null
2    then return 0
3 count ← 1
4 p ← Left-Most-Child(T, p)
5 while p ≠ null
6    do count ← count + CountNodes(T, p)
7       p ← Right-Sibling(T, p)
8 return count
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۱۸۶ و ۱۸۷

یافتن پدر یک گره

```
PARENT(T, r, p)
▷ گره پدر یک گره p را در درخت T به ریشه‌ی r تعیین می‌گردد
▷ در صورتی که p در زیردرخت نباشد پدرش برگردانده نمی‌شود
▷ فرض می‌کنیم که اشاره‌گر به پدر وجود ندارد
1  if p = r
2     then return null
3  q ← LEFT-MOST-CHILD(T, r)
4  while q ≠ null
5     do if p = q
6        then return r
7     s ← PARENT(T, q, p)
8     if s ≠ null
9        then return s
10    q ← RIGHT-SIBLING(T, q)
11 return null
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۸۷ و ۱۸۸

منابع

منابع

- محمد قدسی، داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷.